

Introduction

Every automation platform eventually asks the same question: how do you call an authenticated API without handing your secret to everyone who touches the workflow afterward? In n8n, that question has a very specific failure mode. The HTTP Request node has a plain text field right there in the header configuration, and typing a key directly into it is the path of least resistance. It works immediately. The request succeeds, the demo runs, and nothing about the workflow looks broken.

The problem surfaces later — when that workflow gets exported for backup, checked into a git repo, or shared with a teammate over Slack. At that point the API key travels with it, sitting in plain text inside the JSON. This is not a theoretical risk. It's the default behavior unless you deliberately route secrets through n8n's credential system instead.

This matters beyond n8n specifically. Understanding the three shapes that API authentication takes — API keys, Bearer tokens, and Basic auth — is foundational knowledge for anyone building against external services, whether you're writing Python with `requests`, calling a REST API from a backend service, or wiring up a no-code tool. The mechanics are the same everywhere; only the surface changes.

Problem Overview

You're building a workflow that calls a paid, authenticated API. The API requires proof of identity on every request — some kind of credential attached to the HTTP call. There are three common ways an API asks for that proof:

1. **API key auth** — a static key sent in a custom header or query parameter.
2. **Bearer token auth** — a token sent in the `Authorization` header, prefixed with the word `Bearer`.
3. **Basic auth** — a username and password combined and base64-encoded, sent in the `Authorization` header.

The naive approach is to type the credential directly into the HTTP Request node's parameter field. It authenticates correctly. But that field is just a string, and n8n serializes node parameters into the workflow's exported JSON verbatim. Anywhere that JSON goes, the secret goes with it — version control, backups, shared files, all of it.

The actual problem to solve isn't "how do I authenticate this request." It's "how do I authenticate this request without the secret ever becoming part of the artifact I hand to other people."

Example

Suppose you're calling a weather API that expects an API key in a header called `X-API-Key`.

Naive setup: In the HTTP Request node, under Headers, you add `X-API-Key` with the value `wx_live_9f8a2...` typed directly in.

- Output when you run it: the request succeeds, you get weather data back.
- Output when you export the workflow: the JSON file contains `"value": "wx_live_9f8a2..."` in plain text, sitting inside a file now living in your downloads folder, your teammate's inbox, or your repo's history.

Credential-backed setup: You create a Header Auth credential named `Weather API Key`, set the header name to `X-API-Key` and the value to your key — once, inside the credential form, not the node.

- Output when you run it: identical. The request still succeeds.
- Output when you export the workflow: the JSON contains a reference like `"credentialsId": "Weather API Key"` — no key, just a name pointing at something that lives outside the file.

Same result at execution time, completely different result at export time. That's the entire point.

Intuition

The mistake isn't a knowledge gap about HTTP — most people building these workflows know exactly what a header is and what an `Authorization` value looks like. The mistake is architectural: treating a secret like any other configuration value, when it needs to live somewhere that doesn't get serialized and passed around.

An experienced engineer approaches this by asking a different question than "how do I send this key?" They ask "where does this secret live at rest, and what travels when this artifact moves?" Any time configuration and secrets are stored in the same file, that file becomes something you can't safely export, commit, or share — which defeats half the reason you'd want a portable workflow definition in the first place.

The fix follows directly from that framing: separate the two concerns. Configuration (which credential to use, which header to inject it into) stays in the workflow. The secret itself lives in a separate, encrypted store, referenced by name. n8n's credential system is exactly that separation, and once you see the pattern, it maps onto how every serious system handles secrets — environment variables

instead of hardcoded config, secret managers instead of `.env` files in git, vault services instead of connection strings in code.

Brute Force Solution

Idea: Type the API key, Bearer token, or Basic auth value directly into the relevant field on the HTTP Request node (a header value, or inline in the Authorization header).

Algorithm: 1. Open the HTTP Request node. 2. Add a header (or set Basic Auth fields) with the raw secret value. 3. Save and run.

Advantages: - Fastest possible setup — no credential dialog, no extra clicks. - Fine for a one-off, throwaway test that never leaves your machine.

Disadvantages: - The secret is serialized into the workflow's exported JSON in plain text. - Any export, git commit, backup, or "let me send you this workflow" leaks it. - No central place to rotate the key — if it's compromised, you have to hunt down every workflow that references it. - Time complexity: $O(1)$ to set up. Space complexity: the secret now occupies space in every copy of the file, forever, until manually scrubbed.

Optimal Solution

The optimal approach uses n8n's built-in credential store, which is encrypted and stored separately from workflow definitions.

Step-by-step:

Step	What happens
1	Create a new credential, choosing the type that matches the API: Header Auth (API key), Bearer, or Basic Auth
2	Enter the secret value once, into the credential form — never into the node
3	Give the credential a name (e.g., <code>Weather API Key</code>)
4	In the HTTP Request node, select that credential from a dropdown instead of typing a value
5	At execution time, n8n pulls the secret from the credential store and substitutes it into the outgoing request
6	When you export the workflow, only the credential <i>name</i> is serialized — the secret stays in the store

The three credential types map directly to how the target API expects proof of identity:

- **Header Auth:** you supply a header name and value. Use this for a static API key sent in a custom header (or query param, depending on the API).
- **Bearer:** you supply only the raw token. n8n prepends the literal word `Bearer` when it builds the `Authorization` header — type the token alone, or you'll end up with a broken double-prefixed header.
- **Basic Auth:** you supply a plain username and password in two separate fields. n8n combines and base64-encodes them when building the request; you never encode anything by hand.

The workflow file, at every point in its life — mid-edit, exported, committed, re-shared — contains only the pointer. The secret is materialized exactly once, at execution time, inside n8n's runtime, and never written back into the definition.

Python Code

The same separation-of-concerns pattern applies outside n8n. Here's how it looks calling the same three auth types directly with Python's `requests` library, pulling secrets from environment variables instead of hardcoding them — the same principle, different runtime.

```
import os
import base64
import requests

def call_with_api_key(url: str) -> requests.Response:
    """API key auth: static key in a custom header."""
    api_key = os.environ["WEATHER_API_KEY"] # never hardcoded
    headers = {"X-API-Key": api_key}
    return requests.get(url, headers=headers)

def call_with_bearer_token(url: str) -> requests.Response:
    """Bearer auth: token in the Authorization header, prefixed once."""
    token = os.environ["SERVICE_BEARER_TOKEN"]
    headers = {"Authorization": f"Bearer {token}"}
    return requests.get(url, headers=headers)

def call_with_basic_auth(url: str) -> requests.Response:
    """Basic auth: username:password, base64-encoded automatically."""
    username = os.environ["SERVICE_USERNAME"]
    password = os.environ["SERVICE_PASSWORD"]
    # requests handles the base64 encoding internally
    return requests.get(url, auth=(username, password))
```

Code Walkthrough

- `os.environ["WEATHER_API_KEY"]` — the key is read from the environment at call time, never written into source. This is the Python equivalent of n8n's credential reference: the code contains a *name* (`WEATHER_API_KEY`), not a value.
- `call_with_api_key` builds the header manually because API key placement (header name, query param) varies by provider — there's no universal standard.
- `call_with_bearer_token` constructs the `Bearer <token>` string exactly once. This mirrors the n8n gotcha directly: if you stored `Bearer xyz123` as the token value instead of just `xyz123`, you'd get a broken double-prefixed header.
- `call_with_basic_auth` passes a tuple to `requests ' auth` parameter. The library base64-encodes `username:password` internally — you never touch encoding logic yourself, same as n8n's Basic Auth credential form.
- None of these functions ever return or log the secret, and none of them accept it as a parameter — it's sourced internally, close to where it's used, which limits how far it can accidentally propagate (into logs, stack traces, or a return value that gets serialized somewhere).

Dry Run

Trace `call_with_bearer_token("https://api.example.com/data")` with `SERVICE_BEARER_TOKEN=tok_abc123` set in the environment:

1. `os.environ["SERVICE_BEARER_TOKEN"]` reads `"tok_abc123"`.
2. `headers` becomes `{"Authorization": "Bearer tok_abc123"}`.
3. `requests.get(url, headers=headers)` sends the GET request with that header attached.
4. The API validates `tok_abc123` against its own records and returns a 200 with the requested data.
5. At no point does `tok_abc123` appear anywhere except the outgoing request — not in the function signature, not in a log line, not in a return value.

Compare this to what would happen if the token were hardcoded as a string literal in the function body: it would now exist in source control history permanently, even if removed in a later commit.

Complexity Analysis

- **Time:** $O(1)$ per request — reading an environment variable and attaching a header is constant-time work, identical to the brute-force version. The credential-backed approach costs nothing

extra at execution time.

- **Space:** $O(1)$ in the workflow/source artifact — the artifact stores a name or a reference, not the secret itself, regardless of how many times the credential is reused across nodes or functions.
- **Why this is correct:** correctness here isn't about big-O, it's about invariants. The invariant you want is "the secret only exists in one place, and that place is not the thing that gets copied around." Both the n8n credential store and environment-variable-based config in Python preserve that invariant; inlining the value anywhere breaks it.

Alternative Solutions

For teams beyond a single environment variable or n8n instance, a dedicated secrets manager (AWS Secrets Manager, HashiCorp Vault, Google Secret Manager) adds rotation policies, access auditing, and per-environment scoping on top of the same core pattern. It's worth reaching for once you have multiple services, multiple environments (staging/prod), or compliance requirements around key rotation — n8n's built-in credential store and a `.env` file both solve the "don't hardcode it" problem, but neither gives you rotation history or access logs out of the box.

Edge Cases

- **Empty or missing credential:** if the environment variable or n8n credential doesn't exist, the request should fail loudly (a `KeyError` in Python, a clear error in n8n) rather than silently sending an unauthenticated request.
- **Re-importing a workflow on a new n8n instance:** the workflow imports successfully and still references the credential by name, but since the credential doesn't exist on the new instance, the node errors at execution until you recreate it there. This is expected behavior, not a bug — it's the pointer/secret separation working as designed.
- **Token already includes "Bearer":** typing `Bearer tok_abc123` into a Bearer credential produces `Authorization: Bearer Bearer tok_abc123`, which the API will reject. Store the raw token only.
- **Basic auth with special characters in the password:** base64 encoding handles arbitrary bytes correctly, but if you're building the header manually instead of letting n8n or `requests` do it, forgetting to encode will silently corrupt the header.
- **Basic auth over plain HTTP:** base64 is an encoding, not encryption — it's trivially reversible. Basic auth only offers real protection when the connection itself is HTTPS.

Common Mistakes

1. Typing the secret directly into the HTTP Request node's parameter field instead of a credential, because it's faster for a quick test.
2. Including the word `Bearer` in the token value when the platform already prepends it, producing a double-prefixed header.
3. Assuming Basic auth is inherently less secure than the other two — the risk is the same for all three if the connection isn't HTTPS; base64 is not encryption.
4. Reusing the same credential name across unrelated projects, making rotation confusing when one key needs to change and others don't.
5. Committing a workflow export to version control without checking whether any field still contains a literal secret from before the credential was wired up.
6. Not checking the API's documentation for which `Authorization` scheme it expects before guessing — a username/password pair means Basic auth, a raw token means Bearer, anything else in a custom header usually means API key auth.

Interview Questions

- How would you design a system so that exporting or serializing configuration never leaks a secret?
- What's the difference between encoding and encryption, and why does that matter for Basic auth?
- If a workflow or config file references a credential that doesn't exist in the current environment, what should happen, and why is failing loudly better than failing silently?
- How would you rotate an API key with zero downtime if it's referenced by name across many services?
- Walk through what actually gets sent over the wire for each of the three auth types.

Similar Problems

- **Environment variable management in CI/CD pipelines** — the same secret/config separation problem, solved with pipeline secret stores instead of a credential UI.
- **OAuth2 token refresh flows** — a related but more complex auth shape where the "secret" is short-lived and needs automatic renewal.
- **Secrets scanning in git repositories** (e.g., detecting leaked keys in commit history) — the defensive counterpart to this lesson, for catching leaks after the fact.
- **Database connection string management** — the exact same pattern (pointer in config, secret in a vault) applied to database credentials instead of API auth.

Key Takeaways

- Secrets belong in one place, referenced everywhere else by name — never duplicated into whatever artifact you're about to export or share.
- The three common API auth shapes are API key, Bearer token, and Basic auth; the docs will tell you which one to use if you look for the word "Authorization."
- n8n's credential store enforces this separation automatically once you use it — the discipline is choosing to use it instead of the faster inline field.
- Base64 encoding is not encryption; Basic auth's safety depends entirely on the connection being HTTPS.
- A credential reference failing after an import to a new environment is the system working correctly, not a bug to work around.

Watch the Video

This lesson is easier to follow with the live node setup in front of you — watch the full walkthrough here: {LINK}

About the Series

This is part of **Fun with Learning Technology**, a series that takes the tools developers actually build with — n8n, APIs, databases, and whatever else shows up in a real project — and breaks down not just how to use them, but why they work the way they do. Each lesson picks a real mistake or a real design decision and unpacks it until it clicks.

Call To Action

If this lesson saved you from shipping a leaked key, like the video, subscribe on YouTube for the rest of the series, and subscribe to the Substack for written breakdowns like this one delivered straight to your inbox. Drop a comment with the auth type you got wrong first — every experienced engineer has a story here. Share this with a teammate before their next workflow export.

Quick Review

Trigger for Basic Auth over Bearer/API key?

Legacy/internal API needing username+password on every request, not token-issuing.

What actually travels in an HTTP Bearer Auth header?

'Authorization: Bearer <token>', base64-plain, visible to anyone sniffing unencrypted traffic.

Why does n8n's credential system exist instead of hardcoding keys in nodes?

Keeps secrets encrypted/centralized, reusable across workflows, and out of exported JSON.

Common bug: API key auth request fails with 401 despite correct key.

Key sent in wrong slot — query param vs header vs custom header name mismatch.

API Key auth vs Bearer Token auth — key difference?

API key is a static shared secret in header/query; Bearer token is often short-lived, issued by an auth flow.

Interview: why is Basic Auth considered weak without HTTPS?

Credentials are base64-encoded, not encrypted — trivially decoded if intercepted over plain HTTP.